

Üüriturg — Estonian Rental Market Monitoring Platform

Team Members: Hashim | Sudais | Calvin | Daboikiabo

University of Tartu — Spring 2026

1. Complete System Architecture

The Üüriturg platform is composed of eight microservices, each owning its own domain and database. Services communicate via REST (synchronous) and a RabbitMQ message broker (asynchronous) to remain loosely coupled.

1.1 Service Overview

Service	Responsibilities	Key Relationships
Scraper Service	Scrape KV.ee, City24, Renden; normalize & store listings	Publishes events to Alert Service; read by Analytics & Listing
Analytics Service	Compute price trends, averages, vacancy rates per neighborhood	Reads from Scraper; serves data to Neighborhood & frontend
User Service	Registration, authentication, profile & saved searches	Central identity hub; called by all other services
Alert Service	Alert rules & matching; trigger notifications on match	Consumes Scraper events; calls User & Notification
Neighborhood Service	Neighborhood profiles, amenity scores, resident reviews	Reads from Analytics; references User for reviews
Landlord Service	Landlord profiles, tenant reviews, reputation scores	Calls User; referenced by Listing Service
Listing Service	Landlord-managed listings, claim verification, listing status	References Scraper, User, and Landlord services
Notification Service	Email & in-app message delivery, delivery tracking, templates	Called by Alert, Landlord, and Listing; calls User for contact info

1.2 Architecture Diagram

The architecture diagram is laid out as follows. Each service is represented as a colored box with its name, responsibility summary, and owning team member. Solid arrows represent synchronous REST calls. Dashed arrows represent asynchronous communication via the RabbitMQ message broker. The

diagram is structured in three tiers:

Top tier (data producers): Scraper Service sits at the top center. It is the primary data source that scrapes rental portals and publishes listing.new events to the message broker.

Middle tier (core services): Analytics Service (left), User Service (center), and Alert Service (right) form the middle row. Analytics reads listing data from Scraper via REST. User Service is the central identity hub with arrows coming in from all other services. Alert Service consumes listing.new events from the broker and calls User Service synchronously for contact info, then triggers Notification Service.

Bottom tier (domain-specific services): Neighborhood Service (far left), Landlord Service (center-left), Listing Service (center-right), and Notification Service (far right) form the bottom row. Neighborhood reads analytics data. Landlord calls User for identity. Listing Service calls Scraper, User, and Landlord. Notification Service calls User for recipient info and is triggered by Alert, Landlord, and Listing services.

Message Broker (RabbitMQ): Shown as a purple bar below the top tier. The Scraper publishes listing.new events; the Alert Service consumes them. Additional async events include: listing.claimed (Listing Service publishes, Notification consumes), review.posted (Landlord Service publishes, Notification consumes).

Color coding by owner: Red = Hashim (Scraper, Analytics), Dark blue = Sudais (User, Alert), Green = Calvin (Neighborhood, Landlord), Orange = Daboikiabo (Listing, Notification).

1.3 Scope of Implementation

Service	Owner	Status	Role in System
Scraper Service	Hashim	Will be implemented	Data producer; publishes listing.new events
Analytics Service	Hashim	Will be implemented	Batch aggregation; serves trend data
User Service	Sudais	Will be implemented	Central identity provider; called by all services
Alert Service	Sudais	Will be implemented	Event consumer; triggers notifications
Neighborhood Service	Calvin	Will be implemented	Neighborhood profiles & reviews
Landlord Service	Calvin	Will be implemented	Reputation tracking; tenant reviews
Listing Service	Daboikiabo	Will be implemented	Landlord-managed listings; claim workflow
Notification Service	Daboikiabo	Will be implemented	Cross-cutting notification delivery

All eight services will be implemented. No services are replaced by shared infrastructure components. The message broker (RabbitMQ) is a shared platform-level concern used by multiple services.

2. Service APIs

2.1 Scraper Service (Hashim)

Base URL: /api/scraper

Endpoint	Method	Description
/listings	GET	Retrieve all active listings with optional filters (?neighborhood=, &maxPrice=, &minSize=)
/listings/latest	GET	Retrieve last 50 scraped listings
/listings/{listingId}	GET	Retrieve a single listing by ID
/scraper/trigger	POST	Manually trigger a scrape run (admin/demo)
/scraper/status	GET	Get last scrape time, total listings, job status
/scraper/jobs	GET	List recent scrape job history

2.2 Analytics Service (Hashim)

Base URL: /api/analytics

Endpoint	Method	Description
/analytics/neighborhoods	GET	Average price & price/m ² per neighborhood (current snapshot)
/analytics/trends	GET	Price trend per day for a neighborhood (?neighborhood=&days=30)
/analytics/summary	GET	City-wide stats: total listings, cheapest, most expensive, average price, average size
/analytics/cheapest	GET	Top 10 cheapest listings matching filters (?neighborhood=&maxPrice=)
/analytics/compute	POST	Trigger analytics recomputation (admin/scheduler)

2.3 User Service (Sudais)

Base URL: /api/users

Endpoint	Method	Description
/users	POST	Register a new user (name, email, role)
/users/{userId}	GET	Retrieve a user's full profile by ID
/users/{userId}	PUT	Update user profile and preferences
/users/{userId}	DELETE	Deactivate a user account (admin only)
/users/validate/{userId}	GET	Validate user exists and is active (called internally by other services)
/users/{userId}/searches	GET	Retrieve a user's saved searches
/users/{userId}/searches	POST	Create a new saved search for user

2.4 Alert Service (Sudais)

Base URL: /api/alerts

Endpoint	Method	Description
/alerts	POST	Create a new alert rule (userId, neighborhood, maxPrice, minSize)
/alerts	GET	List all alert rules (admin/demo)

Endpoint	Method	Description
/alerts/{alertId}	GET	Retrieve a specific alert rule
/alerts/{alertId}	DELETE	Deactivate/delete an alert rule
/alerts/{alertId}/matches	GET	List all matches for an alert rule
/alerts/test/{alertId}	POST	Manually test fire an alert (demo)

2.5 Neighborhood Service (Calvin)

Base URL: /api/neighborhoods

Endpoint	Method	Description
/neighborhoods	GET	List all tracked neighborhoods with scores
/neighborhoods/{id}	GET	Retrieve detailed neighborhood profile
/neighborhoods/{id}/reviews	GET	List resident reviews for a neighborhood
/neighborhoods/{id}/reviews	POST	Submit a new neighborhood review (verifies user via User Service)
/neighborhoods/{id}/reviews/{reviewId}	DELETE	Delete a review (admin or author)
/neighborhoods/compare	GET	Compare neighborhoods by price, score (?ids=id1,id2,id3)

2.6 Landlord Service (Calvin)

Base URL: /api/landlords

Endpoint	Method	Description
/landlords	POST	Register a new landlord profile (verifies user via User Service)
/landlords/{landlordId}	GET	Retrieve landlord profile with reputation
/landlords/{landlordId}/reviews	GET	List all tenant reviews for a landlord
/landlords/{landlordId}/reviews	POST	Submit a tenant review for a landlord (verifies reviewer via User Service)
/landlords/{landlordId}/reviews/{reviewId}/respond	PUT	Landlord responds to a tenant review
/landlords/{landlordId}/reputation	GET	Get computed reputation score

2.7 Listing Service (Daboikiabo)

Base URL: /api/managed-listings

Endpoint	Method	Description
/managed-listings	POST	Create a new direct listing (landlord posts property, verifies via User Svc)
/managed-listings	GET	List all managed listings with filters (?status=, &landlordId=)
/managed-listings/{id}	GET	Retrieve a specific managed listing
/managed-listings/{id}	PUT	Update listing details or status (available, rented, withdrawn)
/managed-listings/claims	POST	Submit a claim request for a scraped listing (references Scraper Service listing)
/managed-listings/claims/{claimId}	GET	Check status of a claim request
/managed-listings/claims/{claimId}/approve	PUT	Approve or reject a claim (admin only)

2.8 Notification Service (Daboikiabo)

Base URL: /api/notifications

Endpoint	Method	Description
/notifications	POST	Request delivery of a notification (called by other services)
/notifications/{id}	GET	Retrieve a specific notification record
/notifications/user/{userId}	GET	Retrieve all notifications for a user
/notifications/{id}/status	GET	Check delivery status of a notification
/notifications/templates	GET	List all available message templates
/notifications/templates	POST	Create a new message template (admin)
/notifications/preferences/{userId}	GET	Get notification preferences for user
/notifications/preferences/{userId}	PUT	Update notification preferences

3. Service Data Models

Each microservice manages its own database. No direct database sharing occurs between services. Cross-service references use logical identifiers (UUID) only.

3.1 Scraper Service (Hashim)

Entity	Attributes	Type	Relationships
Listing (Aggregate Root)	listingId (UUID, PK) source (Enum: KV_EE, CITY24, RENDIN) externalId (String) title (String) price (Decimal) size (Decimal, m²) pricePerSqm (Decimal, derived) rooms (Integer) neighborhood (String) address (String) url (String) scrapedAt (DateTime) isActive (Boolean)	Aggregate Root	Referenced by AlertMatch (Alert Service) and ManagedListing (Listing Service) via listingId
ScrapeJob	jobId (UUID, PK) source (Enum) startedAt (DateTime) completedAt (DateTime) status (Enum: RUNNING, COMPLETED, FAILED) listingsFound (Integer) newListings (Integer)	Entity	Produces Listings (1 job -> many listings)

3.2 Analytics Service (Hashim)

Entity	Attributes	Type	Relationships
Neighborhood Snapshot (Aggregate Root)	snapshotId (UUID, PK) neighborhood (String) date (Date) averagePrice (Decimal) averagePricePerSqm (Decimal) medianPrice (Decimal) listingCount (Integer) priceChangePercent (Decimal)	Aggregate Root	References Listing data by neighborhood (no FK). Read by Neighborhood Service.

3.3 User Service (Sudais)

Entity	Attributes	Type	Relationships
User (Aggregate Root)	userId (UUID, PK) firstName (String) lastName (String) email (String) phone (String) role (Enum: TENANT, LANDLORD, ADMIN) createdAt (DateTime) updatedAt (DateTime)	Aggregate Root	Referenced by all other services via userId. One-to-many with SavedSearch.
SavedSearch	searchId (UUID, PK) userId (UUID, FK) neighborhood (String) maxPrice (Decimal) minSize (Decimal) minRooms (Integer) createdAt (DateTime)	Entity	Belongs to User (many-to-one)

3.4 Alert Service (Sudais)

Entity	Attributes	Type	Relationships
AlertRule (Aggregate Root)	alertId (UUID, PK) userId (UUID, ref to User) neighborhood (String, nullable) maxPrice (Decimal) minSize (Decimal, nullable) minRooms (Integer, nullable) isActive (Boolean) createdAt (DateTime)	Aggregate Root	References User.userId (cross-service). One-to-many with AlertMatch.
AlertMatch	matchId (UUID, PK) alertId (UUID, FK) listingId (UUID, ref to Listing) matchedAt (DateTime) notified (Boolean)	Entity	Belongs to AlertRule. References Listing.listingId (cross-service).

3.5 Neighborhood Service (Calvin)

Entity	Attributes	Type	Relationships
Neighborhood (Aggregate Root)	neighborhoodId (UUID, PK) name (String) city (String) description (String) latitude (Double) longitude (Double) amenityScore (Decimal) transportScore (Decimal) safetyRating (Decimal)	Aggregate Root	One-to-many with NeighborhoodReview. References Neighborhood Snapshot (Analytics) by name.
Neighborhood Review	reviewId (UUID, PK) neighborhoodId (UUID, FK) userId (UUID, ref to User) rating (Integer, 1-5) text (String) createdAt (DateTime)	Entity	Belongs to Neighborhood. References User.userId (cross-service).

3.6 Landlord Service (Calvin)

Entity	Attributes	Type	Relationships
LandlordProfile (Aggregate Root)	landlordId (UUID, PK) userId (UUID, ref to User) displayName (String) verificationStatus (Enum: PENDING, VERIFIED, REJECTED) averageRating (Decimal, derived) totalReviews (Integer, derived) createdAt (DateTime)	Aggregate Root	References User.userId (cross-service). One-to-many with TenantReview. Referenced by ManagedListing.
TenantReview	reviewId (UUID, PK) landlordId (UUID, FK) reviewerUserId (UUID, ref User) rating (Integer, 1-5) text (String) landlordResponse (String, null) createdAt (DateTime)	Entity	Belongs to LandlordProfile. References User.userId (cross-service).

3.7 Listing Service (Daboikiabo)

Entity	Attributes	Type	Relationships
ManagedListing (Aggregate Root)	managedListingId (UUID, PK) landlordId (UUID, ref Landlord) scrapedListingId (UUID, nullable, ref Listing) title (String) description (String) price (Decimal) size (Decimal) rooms (Integer) address (String) status (Enum: AVAILABLE, RENTED, WITHDRAWN) isVerified (Boolean) createdAt, updatedAt (DateTime)	Aggregate Root	References Landlord Profile.landlordId and Listing.listingId (cross-service). One-to-one with ClaimRequest.
ClaimRequest	claimId (UUID, PK) landlordId (UUID, FK) scrapedListingId (UUID, ref Listing) status (Enum: PENDING, APPROVED, REJECTED) verificationDetails (String) submittedAt (DateTime) resolvedAt (DateTime)	Entity	Belongs to ManagedListing. References Listing.listingId (cross-service).

3.8 Notification Service (Daboikiabo)

Entity	Attributes	Type	Relationships
Notification (Aggregate Root)	notificationId (UUID, PK) recipientUserId (UUID, ref User) channel (Enum: EMAIL, IN_APP) subject (String) body (String) status (Enum: PENDING, SENT, FAILED) sentAt (DateTime) createdAt (DateTime)	Aggregate Root	References User.userId (cross-service). Uses NotificationTemplate.
Notification Template	templateId (UUID, PK) name (String) subjectTemplate (String) bodyTemplate (String)	Entity	Used by Notification (many-to-one).
Notification Preference	preferenceId (UUID, PK) userId (UUID, ref User) channel (Enum: EMAIL, IN_APP) enabled (Boolean)	Entity	References User.userId (cross-service).

4. System Workflows

Workflow 1: New Listing Alert Notification (Sync + Async)

This workflow demonstrates synchronous REST communication combined with asynchronous event-driven notification. It is triggered when the Scraper Service detects a new rental listing.

Services Involved:

- Scraper Service (producer) — scrapes listings and publishes events
- Alert Service (consumer/orchestrator) — matches listings to alert rules
- User Service (sync call target) — provides user contact info
- Notification Service (sync call target) — delivers the alert notification

Interaction Sequence:

Step	From	To	Action / Event
1	Scheduler	Scraper Service	Scheduled scrape runs; new listings detected and stored in scraper_db
2	Scraper Service	Message Broker	Publishes listing.new event to topic: listing.events Payload: { listingId, title, price, neighborhood, size, rooms }
3	Alert Service	Message Broker	Consumes listing.new event from topic: listing.events [ASYNC]
4	Alert Service	alert_db	Evaluates all active AlertRules against the new listing; creates AlertMatch record for each match
5	Alert Service	User Service	GET /users/validate/{userId} — synchronous REST call to verify alert owner is active [SYNC]
6	User Service	Alert Service	Returns 200 OK { userId, email, role }
7	Alert Service	Notification Service	POST /notifications — synchronous REST call to trigger notification delivery [SYNC]
8	Notification Svc	User Service	GET /users/{userId} — retrieves full contact info for message delivery [SYNC]
9	Notification Svc	notification_db	Creates Notification record (status: sent); delivers email/in-app alert

Data Tables Updated:

- scraper_db — Listing: new listing records inserted
- alert_db — AlertMatch: new match record(s) created (notified = true)
- notification_db — Notification: record created (status: sent)

Event Details:

Event	Topic / Channel	Payload
listing.new	listing.events	{ listingId, title, price, neighborhood, size, rooms, url, scrapedAt }

Sequence Diagram Description: The diagram shows six participants in order from left to right: Scheduler, Scraper Service, Scraper DB, Message Broker, Alert Service, Alert DB, User Service, Notification Service, Notification DB. The scheduler triggers the Scraper. The Scraper stores listings and publishes listing.new to the broker (dashed async arrow). The Alert Service consumes the event, queries its DB for matching rules, calls User Service (solid sync arrow) to validate the user, then calls Notification Service (solid sync arrow) to deliver the alert. Notification Service calls User Service for contact info, then stores the notification record.

Workflow 2: Landlord Listing Claim & Verification (Sync + Async)

This workflow demonstrates a landlord claiming a scraped listing. It combines synchronous identity verification with asynchronous notification delivery.

Services Involved:

- Listing Service (orchestrator) — processes the claim request
- User Service (sync call) — verifies landlord identity and role
- Scraper Service (sync call) — retrieves the original scraped listing data
- Notification Service (async consumer) — notifies landlord of claim result

Step	From	To	Action / Event
1	Landlord (User)	Listing Service	POST /managed-listings/claims — submits claim with scrapedListingId and verificationDetails
2	Listing Service	User Service	GET /users/validate/{userId} — verifies landlord is active and has LANDLORD role [SYNC]
3	User Service	Listing Service	Returns 200 OK { userId, role: LANDLORD }
4	Listing Service	Scraper Service	GET /listings/{scrapedListingId} — retrieves original listing data [SYNC]
5	Scraper Service	Listing Service	Returns 200 OK { listing details }
6	Listing Service	listing_db	Creates ClaimRequest record (status: PENDING); creates ManagedListing record linked to scraped listing
7	Listing Service	Message Broker	Publishes listing.claimed event to topic: listing.events Payload: { claimId, landlordId, status }
8	Notification Svc	Message Broker	Consumes listing.claimed event [ASYNC]
9	Notification Svc	User Service	GET /users/{landlordUserId} — retrieves contact info [SYNC]
10	Notification Svc	notification_db	Creates Notification record; sends claim confirmation to landlord

Data Tables Updated:

- listing_db — ClaimRequest: new record (status: PENDING)
- listing_db — ManagedListing: new record linked to scraped listing
- notification_db — Notification: claim confirmation sent

Event Details:

Event	Topic / Channel	Payload
listing.claimed	listing.events	{ claimId, landlordId, scrapedListingId, status, submittedAt }

Sequence Diagram Description: Six participants left to right: Landlord, Listing Service, Listing DB, User Service, Scraper Service, Message Broker, Notification Service, Notification DB. Landlord sends POST claim. Listing Service makes two sync calls (User Service to validate, Scraper Service to get listing). Then it stores the claim, publishes listing.claimed (dashed async). Notification Service consumes, gets contact info from User Service (sync), and delivers the notification.

Workflow 3: Neighborhood Review Submission (Synchronous)

This workflow demonstrates purely synchronous integration when a user submits a neighborhood review.

Services Involved:

- Neighborhood Service (orchestrator)
- User Service (sync call) — identity verification
- Analytics Service (sync call) — retrieves current price data for the neighborhood

Step	From	To	Action / Event
1	User	Neighborhood Svc	POST /neighborhoods/{id}/reviews — submits review with rating and text
2	Neighborhood Svc	User Service	GET /users/validate/{userId} — verifies user is active [SYNC]
3	User Service	Neighborhood Svc	Returns 200 OK { userId, role }
4	Neighborhood Svc	neighborhood_db	Creates NeighborhoodReview record; recalculates neighborhood scores
5	Neighborhood Svc	Analytics Service	GET /analytics/neighborhoods — retrieves current price data [SYNC]
6	Analytics Service	Neighborhood Svc	Returns price snapshot for neighborhood
7	Neighborhood Svc	User	Returns 201 Created { reviewId, updated neighborhood profile }

Data Tables Updated:

- neighborhood_db — NeighborhoodReview: new record created
- neighborhood_db — Neighborhood: scores recalculated

Sequence Diagram Description: Four participants: User, Neighborhood Service, Neighborhood DB, User Service, Analytics Service. All arrows are solid (synchronous REST). User sends POST review. Neighborhood Service validates user, stores the review, fetches analytics, and returns the enriched response.

Workflow 4: Analytics Recomputation from Scraper Data (Synchronous)

This workflow shows how the Analytics Service reads listing data from the Scraper Service and computes daily neighborhood snapshots.

Step	From	To	Action / Event
1	Scheduler/Admin	Analytics Service	POST /analytics/compute — triggers daily recomputation
2	Analytics Service	Scraper Service	GET /listings?neighborhood=X — retrieves all active listings per neighborhood [SYNC]
3	Scraper Service	Analytics Service	Returns listing data for aggregation
4	Analytics Service	analytics_db	Computes averagePrice, medianPrice, pricePerSqm, listingCount per neighborhood. Creates/updates NeighborhoodSnapshot records.
5	Analytics Service	Scheduler/Admin	Returns 200 OK { snapshotsCreated }

Data Tables Updated:

- analytics_db — NeighborhoodSnapshot: new daily snapshot records created/updated

5. Integration Mechanisms

5.1 Synchronous Communication (REST)

Services communicate synchronously using Spring WebClient (reactive HTTP client). All REST calls include proper error handling and timeouts.

Caller	Target	Endpoint	Purpose
Alert Service	User Service	GET /users/validate/{id}	Verify alert owner is active
Alert Service	Notification Svc	POST /notifications	Trigger alert notification
Analytics Service	Scraper Service	GET /listings	Read listing data for aggregation
Neighborhood Svc	User Service	GET /users/validate/{id}	Verify reviewer identity
Neighborhood Svc	Analytics Service	GET /analytics/neighborhoods	Read price data for profiles
Landlord Service	User Service	GET /users/validate/{id}	Verify landlord identity
Listing Service	User Service	GET /users/validate/{id}	Verify landlord role
Listing Service	Scraper Service	GET /listings/{id}	Retrieve scraped listing for claim
Listing Service	Landlord Service	GET /landlords/{id}	Retrieve landlord profile
Notification Svc	User Service	GET /users/{id}	Retrieve contact info for delivery

5.2 Asynchronous Communication (RabbitMQ)

Events are published to a RabbitMQ message broker using topic exchanges. This decouples producers from consumers and improves system resilience.

Event	Topic / Channel	Producer	Consumer(s)
listing.new	listing.events	Scraper Service	Alert Service
listing.claimed	listing.events	Listing Service	Notification Service
review.posted	landlord.events	Landlord Service	Notification Service

6. Implementation Responsibilities

Hashim

I am responsible for implementing the following two microservices:

Scraper Service: Full REST API (6 endpoints) + own persistent database (scraper_db). Acts as the primary data source for the entire system. Publishes listing.new events to RabbitMQ (listing.events topic) consumed by Alert Service. Called synchronously by Analytics Service (for listing data) and Listing Service (for claim verification). Participates in Workflow 1 (event producer) and Workflow 4 (data source).

Analytics Service: Full REST API (5 endpoints) + own persistent database (analytics_db). Reads listing data from Scraper Service via synchronous REST calls. Computes daily neighborhood snapshots. Called synchronously by Neighborhood Service for price data display. Participates in Workflow 3 (data provider) and Workflow 4 (primary actor).

Both services expose a non-trivial REST API, manage their own persistent data, participate in at least two interactions with other services, and are part of the implemented system workflows.

Sudais

I am responsible for implementing the following two microservices:

User Service: Full REST API (7 endpoints) + own persistent database (user_db). Acts as the central identity provider; called synchronously by all other services for user validation and contact info retrieval. Participates in all four workflows as the identity verification target.

Alert Service: Full REST API (6 endpoints) + own persistent database (alert_db). Consumes listing.new events from RabbitMQ (listing.events topic) published by Scraper Service. Calls User Service synchronously for alert owner validation and Notification Service for alert delivery. Primary orchestrator in Workflow 1.

Both services expose a non-trivial REST API, manage their own persistent data, participate in at least two interactions with other services, and are part of the implemented system workflows.

Calvin

I am responsible for implementing the following two microservices:

Neighborhood Service: Full REST API (6 endpoints) + own persistent database (neighborhood_db). Manages neighborhood profiles and resident reviews. Calls User Service synchronously for reviewer identity verification and Analytics Service for price data. Primary orchestrator in Workflow 3.

Landlord Service: Full REST API (6 endpoints) + own persistent database (landlord_db). Manages landlord profiles and tenant reviews. Calls User Service synchronously for identity verification. Publishes review.posted events to landlord.events topic consumed by Notification Service. Referenced by Listing Service for landlord profile data.

Both services expose a non-trivial REST API, manage their own persistent data, participate in at least two interactions with other services, and are part of the implemented system workflows.

Daboikiabo

I am responsible for implementing the following two microservices:

Listing Service: Full REST API (7 endpoints) + own persistent database (listing_mgmt_db). Manages landlord-owned listings and the claim workflow. Calls User Service, Scraper Service, and Landlord Service

synchronously. Publishes listing.claimed events to listing.events topic. Primary orchestrator in Workflow 2.

Notification Service: Full REST API (8 endpoints) + own persistent database (notification_db). Cross-cutting notification delivery service. Consumes listing.claimed events from Listing Service and review.posted events from Landlord Service via RabbitMQ. Calls User Service synchronously for recipient contact info. Called by Alert Service in Workflow 1. Participates in Workflow 2 as the async consumer.

Both services expose a non-trivial REST API, manage their own persistent data, participate in at least two interactions with other services, and are part of the implemented system workflows.

6.1 Workflows to Be Implemented

Workflow	Integration Style	Services Involved
Workflow 1: New Listing Alert Notification	Sync + Async	Scraper, Alert, User, Notification
Workflow 2: Landlord Listing Claim & Verification	Sync + Async	Listing, User, Scraper, Notification
Workflow 3: Neighborhood Review Submission	Sync only	Neighborhood, User, Analytics
Workflow 4: Analytics Recomputation	Sync only	Analytics, Scraper

Services that will be implemented: All eight business services listed in Assignment 1 (Scraper, Analytics, User, Alert, Neighborhood, Landlord, Listing, Notification). No services are replaced by shared components. The RabbitMQ message broker is a shared platform-level concern used by the system.